Assignment-8

Name:A.Naga Rishik Reddy

Hall Ticket:2303A51089

Batch-02

Task Description #1 (Username Validator – Apply AI in

Authentication Context)

```python
#function to validate username
def is_valid_username(username):
    if len(username) < 5 or len(username) > 15:
        return False
    if not username[0].isalpha():
        return False
    for char in username:
        if not (char.isalnum() or char == '_'):
            return False
    return True
#assert test cases for is_valid_username function
assert is_valid_username("user123") == True
assert is_valid_username("12user") == False
assert is_valid_username("us er") == False
assert is_valid_username("validuser") == True
assert is_valid_username("user") == False
print("All test cases passed succesfully!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted> & C:\Users\Rishi\anaconda\python.exe "c:/Users/Rishi/Documents/Naga Ri
shik/AI assisted/Assignment-1.py"
All test cases passed succesfully!
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted>
```

Activate Windows
Go to Settings to activate

Task Description #2 (Even–Odd & Type Classification – Apply

AI for Robust Input Handling)

```python
#function to classify value
def classify_value(x):
    if isinstance(x, int):
        if x == 0:
            return "Zero"
        elif x % 2 == 0:
            return "Even"
        else:
            return "Odd"
    else:
        return "Invalid input"
#assert test cases for classify_value function
assert classify_value(8) == "Even"
assert classify_value(7) == "Odd"
assert classify_value("abc") == "Invalid input"
assert classify_value(0) == "Zero"
print("All test cases passed!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted> & C:\Users\Rishi\anaconda\python.exe "c:/Users/Rishi/Documents/Naga Ri
shik/AI assisted/Assignment-1.py"
shik/AI assisted/Assignment-1.py"
All test cases passed!
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted>
```

Activate Windows
Go to Settings to activat

Task Description #3 (Palindrome Checker – Apply AI for

String Normalization)

```python
#function to check if a string is a palindrome
def is_palindrome(text):
    #remove spaces and convert to lowercase
    text = text.replace(" ", "").lower()

    #check if the string is equal to its reverse
    return text == text[::-1]
#assert test cases for is_palindrome function
assert is_palindrome("Madam") == True
assert is_palindrome("A man a plan a canal Panama") == True
assert is_palindrome("Hello") == False
print("All test cases passed!")
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

> & C:\Users\Rishi\anaconda\python.exe "c:/Users/Rishi/Documents/Naga Ri
shik/AI assisted/Assignment-1.py"
All test cases passed!
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted>
```

Activate Windows
Go to Settings to activate Windows.

Task Description #4 (Email ID Validation – Apply AI for Data Validation)

```python
#function to validate email
def validate_email(email):
    if '@' not in email or'.' not in email:
        return False
    if email[0] in '@' or email[-1] in '@':
        return False
    if email[0] in '.' or email[-1] in '.':
        return False
    if email.count('@') !=1 or email.count('.') < 1:
        return False
    if email.index('@') > email.rindex('.'):
        return False
    if ' ' in email:
        return False
    if '@.' in email:
        return False
    return True
#assert test cases for validate_email function
assert validate_email("user@gmail.com") == True
assert validate_email("user@") == False
assert validate_email("@gmail.com") == False
assert validate_email("user@gmail") == False
assert validate_email("usergmail.com") == False
assert validate_email("user@.com") == False
assert validate_email("user@com.") == False
print("All test cases passed for validate_email function.")
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

shik/AI assisted/Assignment-1.py"
All test cases passed for validate_email function.
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted>
```

Activate Windows
Go to Settings to activate Windows.

Task 5 (Perfect Number Checker – Test Case Design)

```python
#function to check if a number is a perfect number
def is_perfect_number(n):
    if n<=1:
        return False
    divisors = []
    for i in range(1, n):
        if n % i == 0:
            divisors.append(i)
    return sum(divisors) == n
#assert test cases for os_perfect_number function
assert is_perfect_number(6) == True
assert is_perfect_number(28) == True
assert is_perfect_number(12) == False
assert is_perfect_number(1) == False
assert is_perfect_number(0) == False
assert is_perfect_number(-6) == False
print("All test cases passed!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted> & C:\Users\Rishi\anaconda\python.exe "c:/Users/Rishi/Documents/Naga Ri
shik/AI assisted/Assignment-1.py"
All test cases passed!
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted>
```

Activate Windows
Go to Settings to activate Win

Ln 18, Col 1 Spaces: 4 UTF-8 CRLF {} Python base (3.13.5)

Task 6 (Abundant Number Checker – Test Case Design)

```python
import unittest
def is_abudant_number(n):
    if n<=1:
        return False
    divisors = []
    for i in range(1, n):
        if n % i == 0:
            divisors.append(i)
    return sum(divisors) > n
class TestAbundantNumber(unittest.TestCase):
    def test_normal_cases(self):
        self.assertTrue(is_abudant_number(12))
        self.assertTrue(is_abudant_number(18))
        self.assertFalse(is_abudant_number(28))
    def test_edge_cases(self):
        self.assertFalse(is_abudant_number(1))
        self.assertFalse(is_abudant_number(0))
        self.assertFalse(is_abudant_number(-5))
    def test_negative_cases(self):
        self.assertFalse(is_abudant_number(-10))
        self.assertFalse(is_abudant_number(-1))
    def test_large_numbers(self):
        self.assertTrue(is_abudant_number(945))
        self.assertFalse(is_abudant_number(496))
if __name__ == '__main__':
            unittest.main()
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

----------------------------------------------------------------------
shik/AI assisted/Assignment-1.py"
....
----------------------------------------------------------------------
----------------------------------------------------------------------
Ran 4 tests in 0.002s

OK
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted>
```

Task 7 (Deficient Number Checker – Test Case Design)

```python
def is_deficient_number(n):
    if n<=1:
        return False
    divisors = []
    for i in range(1, n):
        if n % i == 0:
            divisors.append(i)
    sum_of_divisors = sum(divisors)
    return sum_of_divisors < n
#assert test cases for is_deficient_number function
assert is_deficient_number(8) == True
assert is_deficient_number(12) == False
assert is_deficient_number(15) == True
assert is_deficient_number(-5) == False
assert is_deficient_number(1) == False
print("All test cases passed!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
OK
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted> & C:\Users\Rishi\anaconda\python.exe "c:/Users/Rishi/Documents/Naga Ri
shik/AI assisted/Assignment-1.py"
All test cases passed!
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted>
```

Activate Windows

Task 8 :

Write a function LeapYearChecker and validate its implementation

using 10 pytest test cases

```python
def is_leap_year(year):
    if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
        return True
    else:
        return False
#assert test cases for is_leap_year function
assert is_leap_year(2020) == True, "Test case 1 failed"
assert is_leap_year(2021) == False, "Test case 2 failed"
assert is_leap_year(1900) == False, "Test case 3 failed"
assert is_leap_year(2000) == True, "Test case 4 failed"
print("All test cases passed for is_leap_year function.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
shik/AI assisted/Assignment-1.py"
All test cases passed for is_leap_year function.
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted>
```

Activate Windows

Task 9 :

Write a function SumOfDigits and validate its implementation

using 7 pytest test cases.

```python
'''
write a function sumofdigits and validate its implementation
using 7 pytest test cases.
'''
def sum_of_digits(n):
    if n < 0:
        return None
    return sum(int(digit) for digit in str(n))
#assert sum_of_digits function
assert sum_of_digits(123) == 6
assert sum_of_digits(0) == 0
assert sum_of_digits(999) == 27
assert sum_of_digits(-1) == None
print("All test cases passed!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
shik/AI assisted/Assignment-1.py"
All test cases passed for is_leap_year function.
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted> & C:\Users\Rishi\anaconda\python.exe "c:/Users/Rishi/Documents/Naga Ri
shik/AI assisted/Assignment-1.py"
All test cases passed!
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted>
```

powershell
Python

Activate Windows
Go to Settings to activate Windows.

Task 10 :

Write a function SortNumbers (implement bubble sort) and validate

its implementation using 25 pytest test cases.

```python
'''
write a function sortnumbers (implement bubble sort) and validate
its implementation using 25 pytest test cases
'''
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
# Test cases
def test_bubble_sort():
    assert bubble_sort([5, 2, 9, 1, 5, 6]) == [1, 2, 5, 5, 6, 9]
    assert bubble_sort([]) == []
    assert bubble_sort([1]) == [1]
    assert bubble_sort([3, 2, 1]) == [1, 2, 3]
    assert bubble_sort([1, 2, 3]) == [1, 2, 3]
    assert bubble_sort([5, -1, 3]) == [-1, 3, 5]
    assert bubble_sort([0, -5, -10]) == [-10, -5, 0]
    assert bubble_sort([10, 9, 8]) == [8, 9, 10]
    assert bubble_sort([4, 4, 4]) == [4, 4, 4]
    assert bubble_sort([1, 2, 3, 4, 5]) == [1, 2, 3, 4, 5]
    assert bubble_sort([5, 4, 3, 2, 1]) == [1, 2, 3, 4, 5]
print("All test cases passed!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
shik/AI assisted/Assignment-1.py"
All test cases passed!
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted>
```

Ln 25, Col 1 Spaces: 4 UTF-8 CRLF

Task 11 :

Write a function ReverseString and validate its implementation

using 5 unittest test cases

```python
'''
write a function reversestring and validate its implementation
using 5 unittest test cases
'''
def reversestring(s):
    return s[::-1]
#assert test cases for reverse_string function
assert reversestring("hello") == "olleh", "Test case 1 failed"
assert reversestring("world") == "dlrow", "Test case 2 failed"
assert reversestring("python") == "nohtyp", "Test case 3 failed"
assert reversestring("assignment") == "tnemngissa", "Test case 4 failed"
assert reversestring("reverse") == "esrever", "Test case 5 failed"
print("All test cases passed!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
AssertionError: Test case 4 failed
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted> & C:\Users\Rishi\anaconda\python.exe "c:/Users/Rishi/Documents/Naga Ri
shik/AI assisted/Assignment-1.py"
All test cases passed!
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted>
```

powershell
Python

Activate Windows
Go to Settings to activate Windows.

Task 12 :

Write a function AnagramChecker and validate its implementation

using 10 unittest test cases.

```python
'''
write a function anagramchecker and validate its implementation
using 10 unittest test cases
'''
def anagramchecker(str1, str2):
    return sorted(str1.replace(" ", "").lower()) == sorted(str2.replace(" ", "").lower())

#assert test cases
assert anagramchecker("listen", "silent") == True
assert anagramchecker("triangle", "integral") == True
assert anagramchecker("apple", "pabble") == False
assert anagramchecker("Dormitory", "Dirty Room") == True
assert anagramchecker("School Master", "The Classroom") == True
assert anagramchecker("Astronomer", "Moon Starer") == True
assert anagramchecker("Conversation", "Voices Rant On") == True
assert anagramchecker("The Eyes", "They See") == True
assert anagramchecker("A gentleman", "Elegant man") == True
assert anagramchecker("Clint Eastwood", "Old West Action") == True
print("All test cases passed!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
    return sorted(str.replace(" ", "").lower()) == sorted(str2.replace(" ", "").lower())
                  ^^^
```

```python
'''
write a function armstrongchecker and validate its implementation
using 8 unittest cases
'''
def is_armstrong_number(n):
    num_str = str(n)
    num_digits = len(num_str)
    armstrong_sum = sum(int(digit) ** num_digits for digit in num_str)
    return armstrong_sum == n
#assert test cases
assert is_armstrong_number(153) == True, "Test case 1 failed"
assert is_armstrong_number(9474) == True, "Test case 2 failed"
assert is_armstrong_number(9475) == False, "Test case 3 failed"
assert is_armstrong_number(0) == True, "Test case 4 failed"
assert is_armstrong_number(1) == True, "Test case 5 failed"
assert is_armstrong_number(370) == True, "Test case 6 failed"
assert is_armstrong_number(371) == True, "Test case 7 failed"
assert is_armstrong_number(407) == True, "Test case 8 failed"
print("All test cases passed!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted> & C:\Users\Rishi\anaconda\python.exe "c:/Users/Rishi/Documents/Naga Ri
shik/AI assisted/Assignment-1.py"
All test cases passed!
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted> & C:\Users\Rishi\anaconda\python.exe "c:/Users/Rishi/Documents/Naga Ri
shik/AI assisted/Assignment-1.py"
All test cases passed!
PS C:\Users\Rishi\Documents\Naga Rishik\AI assisted>
```

Activate Windows
Go to Settings to activate W